

# Where does obfuscation happen?

The transformation stage determines which optimizations come afterwards.

| Layer         | Examples                 | What changes?                               | What follows?                                  |
|---------------|--------------------------|---------------------------------------------|------------------------------------------------|
| C source      | Tigress                  | Expressions and control flow in generated C | The native compiler can reshape the result.    |
| LLVM IR       | OLLVM / Hikari / Polaris | IR operations, blocks and edges             | LLVM optimization and instruction selection.   |
| X86 backend   | Polaris backend-obfu     | Machine instructions after IR               | Native changes may be invisible in IR metrics. |
| Linked binary | BinProtect               | Instructions in an existing PE              | No later compiler optimization in these tests. |

Compare the final machine code, not just the representation where obfuscation was added.

Polaris has separate IR and backend paths. BinProtect outputs were semantically invalid in the retained tests.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

Windows x64 | Retained samples only | Source keys in SOURCES.md

# Five obfuscators, five useful lessons

A compact comparison of the tested implementations, not a universal ranking.

| Obfuscator | Best teaching contrast                           | Original PE status             | Main qualification                               |
|------------|--------------------------------------------------|--------------------------------|--------------------------------------------------|
| OLLVM-16   | Classic SUB; working BCF and FLA rewrites        | PASS<br>Selected originals     | Global pass selection in the retained matrix.    |
| Hikari     | Partial SUB coverage; FLA transfer               | PASS<br>Selected originals     | SUB/BCF target effects disappear by O1.          |
| Polaris    | Strong MBA; BCF transfer; wrong FLA rewrite      | PASS<br>Selected originals     | Original PASS does not validate a later rewrite. |
| Tigress    | Source-level effects survive native optimization | PASS<br>Matched environment    | Preserve exact generated C and initialization.   |
| BinProtect | Post-link arithmetic and state dispatch          | INVALID<br>0 / 8 valid outputs | Crash/hang: static observations only.            |

An obfuscator's original output and a deobfuscator's rewritten output need separate validation.

PASS means the retained checks passed. It is not a guarantee for arbitrary programs or inputs.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

# Arithmetic: complexity can shrink or survive

Optimization acts on the obfuscated program before we inspect its machine code.

| Treatment                   | At O0                                                | At higher optimization                                    | Main lesson                                             |
|-----------------------------|------------------------------------------------------|-----------------------------------------------------------|---------------------------------------------------------|
| OLLVM SUB                   | 24 IR sites transformed; native XOR/SUB recognizable | <b>WEAKENED</b><br>O1+: all 24 native operations reappear | Large IR trees need not survive code generation.        |
| Hikari SUB                  | Partial coverage; 213 native instructions            | <b>REMOVED</b><br>O1+: measured target matches clean      | Coverage and optimization survival are different.       |
| Polaris SUB                 | XOR 8/8; ADD 8/8; SUB 3/8 transformed                | <b>WEAKENED</b><br>O1+: native operations recognizable    | Classic SUB is not the stronger MBA treatment.          |
| Tigress<br>EncodeArithmetic | 426 native instructions                              | <b>PERSISTS</b><br>/Ox: 226 vs 106 clean                  | Exact same generated C; substantial complexity remains. |

IR expansion, native instruction count and semantic difficulty are different measurements.

Target: demo\_substitution. Tigress uses MSVC /Ox (recorded as O3); the 106-instruction clean is also /Ox.

Validated Qualified / incomplete Wrong / invalid Observation Not produced

Windows x64 | Retained samples only | Source keys in SOURCES.md

# Opaque flow: what survives optimization?

BCF = bogus control flow. The observation is about the measured native target.

| Treatment         | At O0                                | O1 through O3 / Ox                                      | Interpretation                                   |
|-------------------|--------------------------------------|---------------------------------------------------------|--------------------------------------------------|
| OLLVM BCF         | Clones and opaque branches           | <b>RESIDUE</b><br>Small optimized opaque-loop structure | Optimization removes much, not everything.       |
| Hikari BCF        | Injected native gates already folded | <b>REMOVED</b><br>Measured target matches clean         | IR injection does not imply a native predicate.  |
| Polaris BCF       | Strong cloned/opaque topology        | <b>REMOVED</b><br>Measured target matches clean         | A strong O0 example can be weak at O1+.          |
| Tigress AddOpaque | Initialization-dependent opaque flow | <b>PERSISTS</b><br>/Ox: 6 conditional jumps vs 1 clean  | Runtime state matters to proof and optimization. |

Choose the optimization level deliberately: it can erase the phenomenon being taught.

Scope: demo\_bogus\_control\_flow in the retained optimization study. Jump counts alone do not prove opacity.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

Windows x64 | Retained samples only | Source keys in SOURCES.md

# Flattening can survive without an indirect jump

FLA = control-flow flattening. Look for state-driven dispatch, not one opcode.

| Treatment       | At O0                   | At higher optimization                           | What to look for                              |
|-----------------|-------------------------|--------------------------------------------------|-----------------------------------------------|
| OLLVM FLA       | State/switch dispatcher | Strong flattening persists                       | Repeated state tests and returns to dispatch. |
| Hikari FLA      | State/switch dispatcher | Strong flattening persists                       | Dispatcher structure despite simpler blocks.  |
| Polaris FLA     | Strong state dispatcher | Smaller, but still flattened                     | Simplification is not full unflattening.      |
| Tigress Flatten | Indirect jump table     | O1: direct network<br>O2 and /Ox: indirect table | Same generated C, different native dispatch.  |

A direct chain of comparisons can implement the same flattened state machine as a jump table.

Scope: demo\_flattening. Survival of obfuscation does not establish compatibility with an unflattener.

Validated Qualified / incomplete Wrong / invalid Observation Not produced

# What does each deobfuscator actually produce?

Expression recovery, a branch proof, LLVM IR and a rewritten PE are different outputs.

| Workflow                     | Input / idea                          | Delivered result        | Boundary to remember                    |
|------------------------------|---------------------------------------|-------------------------|-----------------------------------------|
| <b>msynth AST/oracle</b>     | Extracted expression; rewrite / match | Simpler expression      | No rewritten PE in this study.          |
| <b>SiMBA</b>                 | Algebraic MBA simplification          | Simpler expression      | Partial reduction is not full recovery. |
| <b>Synthesizer</b>           | Search for a small expression         | Candidate expression    | Accept only after equivalence proof.    |
| <b>ollvm-unflattener</b>     | Discover states; recover successors   | Rewritten PE or failure | A produced PE can have wrong semantics. |
| <b>Triton DSE/SMT</b>        | Symbolic branch conditions            | Proof / witness         | Analysis itself does not patch the PE.  |
| <b>Proof-driven rewriter</b> | Consume proofs and exact bytes        | Patched PE              | Predicates and dead bytes can remain.   |
| <b>Mergen</b>                | Lift x64 instructions                 | LLVM IR                 | Valid IR can silently miscompute.       |
| <b>Ordinary LLVM O2</b>      | Optimize retained or lifted IR        | Optimized IR            | Cannot repair an unsound lift.          |

**Ask two questions: what representation was recovered, and how was its correctness checked?**

PE = Windows Portable Executable. The Mergen test harness is not a rewritten original PE.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

Windows x64 | Retained samples only | Source keys in SOURCES.md

# From obfuscated code to a working rewrite

All eight inputs below are runnable originals; the output claims differ sharply.

| Case               | Recovered / demonstrated            | Rewritten original PE | Qualification                             |
|--------------------|-------------------------------------|-----------------------|-------------------------------------------|
| OLLVM SUB          | Selected local expressions          | NOT PRODUCED          | msynth recovery is not binary rewriting.  |
| Polaris MBA        | Local recovery; passing Mergen lift | NOT PRODUCED          | Substantial MBA remains in lifted IR.     |
| OLLVM BCF O0       | 22 proved gates patched             | PASS                  | Predicates and dead clone bytes remain.   |
| Polaris BCF O0     | 16 proved gates patched             | PASS                  | Same generic proof workflow transfers.    |
| OLLVM FLA O0       | 25/25 successor pairs               | PASS                  | Graph need not match clean exactly.       |
| Hikari FLA O0      | 28/28 successor pairs               | PASS                  | O0 success does not extend to O2 here.    |
| Polaris FLA O0     | 14/22 true pairs + 1 false pair     | WRONG SEMANTICS       | Legitimate switch cases collapse.         |
| Tigress Flatten O0 | Stock discovery incomplete          | NO OUTPUT             | Fails before symbolic successor analysis. |

A semantic result is not automatically a working executable, and a valid PE is not automatically correct.

PASS refers to retained validation. Successor-pair scores are not universal proofs of program equivalence.

Validated Qualified / incomplete Wrong / invalid Observation Not produced

# BCF: proof becomes a working PE patch

The same generic proof-and-patch workflow worked across two implementations.

| O0 target   | Proved / patched gates | Reachable CFG blocks                    | Retained validation                 |
|-------------|------------------------|-----------------------------------------|-------------------------------------|
| OLLVM BCF   | 22 / 22                | 11 clean<br>70 original<br>37 rewritten | PASS<br>4,591 triples; 0 mismatches |
| Polaris BCF | 16 / 16                | 11 clean<br>54 original<br>30 rewritten | PASS<br>4,598 triples; 0 mismatches |

JNE becomes JMP + NOP. Reachable flow shrinks; predicate arithmetic and dead bytes stay in the PE.

A triple compares clean/original/rewrite. Dedicated PE CFG audit: calls do not split blocks; RET is terminal.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

Windows x64 | Retained samples only | Source keys in SOURCES.md

# Unflattening: the smaller graph can be wrong

Stock llvm-unflattener results on four O0 implementations.

| Input           | Block counts                    | What happened?                                        | Outcome                        |
|-----------------|---------------------------------|-------------------------------------------------------|--------------------------------|
| OLLVM FLA       | 60 flattened<br>17 rewritten    | 25/25 semantic successor pairs recovered              | <b>PASS</b><br>Working rewrite |
| Hikari FLA      | 66 flattened<br>20 rewritten    | 28/28 semantic successor pairs recovered              | <b>PASS</b><br>Working rewrite |
| Polaris FLA     | 56 flattened<br>10 rewritten    | Multiple legitimate switch cases now run sequentially | <b>FAIL</b><br>Wrong result    |
| Tigress Flatten | 38 actual<br>6 stock-discovered | Indirect jump-table discovery is incomplete           | <b>BLOCKED</b><br>No rewrite   |

Visual simplification is not proof of correct deobfuscation.

Miasm CFG counts split at calls. Tigress numbers compare reference vs discovery, NOT a before/after rewrite.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

# Four tempting conclusions to avoid

Keep visual appearance, representation validity and semantic correctness separate.

| Tempting conclusion                       | Retained counterexample         | What the evidence says                                                    |
|-------------------------------------------|---------------------------------|---------------------------------------------------------------------------|
| Smaller CFG = correct                     | Polaris FLA: 56 to 10 blocks    | <b>WRONG</b><br>F9CD8332 instead of DBEFFCE7; exit 1.                     |
| Valid IR = faithful lift                  | Mergen on valid OLLVM SUB input | <b>WRONG</b><br>3,917 / 4,118 mismatches in every lifted stage.           |
| Six discovered blocks = six actual blocks | Tigress Flatten                 | <b>INCOMPLETE</b><br>The jump-table-aware reference has 38 blocks.        |
| Matching PDB ID = valid instruction map   | Rewritten BCF / FLA PEs         | <b>STALE METADATA</b><br>Old RSDS references survive. Use exports / RVAs. |

Validate the output at the level of the claim: expression, CFG, lifted function or complete executable.

Original OLLVM SUB passes native checks. FLA whole-function rewriting moves internal instructions.

Validated    Qualified / incomplete    Wrong / invalid    Observation    Not produced

# A seed is not always a reconstruction key

Archive what actually ran, not only the command that was meant to reproduce it.

| Tested lane                | Repeat observation                                                          | What to retain                                               |
|----------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------|
| OLLVM / Hikari SUB         | <b>REPEATED IDENTICALLY</b><br>Seeded IR in the retained probe              | Exact flags, compiler identity, IR and PE hashes.            |
| Polaris selected IR passes | <b>REPEATED IDENTICALLY</b><br>SUB/BCF/FLA/indirection/gvenc smoke variants | Exact artifacts; not a universal seed guarantee.             |
| Polaris MBA / backend      | <b>VARIED</b><br>Same requested RNG seed was insufficient                   | The exact generated artifact is the specimen.                |
| Tigress                    | <b>WHOLE FILES VARIED</b><br>Identifier salts / ordering were not pinned    | Exact generated C, then reuse it across optimization levels. |
| BinProtect                 | <b>NO EXPOSED SEED</b><br>Outputs varied                                    | Exact PE for static analysis; validity failed here.          |

Reproducibility has several levels: same source, same generated representation and same executable bytes.

Tigress target bodies were more stable than whole generated files; whole-file variation is not proof of semantic variation.

Validated   Qualified / incomplete   Wrong / invalid   Observation   Not produced

# Six examples worth opening in IDA

Use the evidence-pack walkthrough for exact binaries, hashes and RVAs.

| Story            | Target             | What to compare                             | Teaching point                                          |
|------------------|--------------------|---------------------------------------------|---------------------------------------------------------|
| 01 - OLLVM SUB   | Substitution       | Clean vs SUB; native code vs IR             | Backend canonicalization can undo apparent complexity.  |
| 02 - Polaris MBA | Substitution       | 115 vs 678 native instructions              | Large arithmetic expansion without many CFG blocks.     |
| 03 - OLLVM BCF   | Bogus control flow | Clean / original / Triton rewrite           | Proof-driven removal of impossible edges.               |
| 04 - Polaris BCF | Bogus control flow | Same workflow, another implementation       | Semantic transfer is stronger than one signature match. |
| 05 - OLLVM FLA   | Flattening         | Dispatcher vs working stock rewrite         | Recovered semantics need not reproduce the clean graph. |
| 06 - Polaris FLA | Flattening         | Original vs explicitly marked wrong rewrite | A cleaner-looking graph can compute the wrong answer.   |

Start with matched clean binaries. For every rewrite, navigate by export / RVA and distrust old PDB mappings.

Export names: demo\_substitution, demo\_bogus\_control\_flow, demo\_flattening. Story 06 rewrite is failure evidence.

Validated Qualified / incomplete Wrong / invalid Observation Not produced